random_undersampler Â¶

Implementation of the history-preserving random undersampling strategy.

LOGGER `module-attribute` Â¶

```
LOGGER = getLogger(__name__)
```

RandomUndersampler Â¶

```
RandomUndersampler(fraction: float | None = None, n: int | None = None, n_positives: int | None = None, target_field: str | None = None, target_value: TargetType | set[TargetType] | None = None, seed: int | None = None)
```

Bases: `BaseSampler`

History-preserving random undersampling strategy.

The random undersampling is performed according to one (and only one) of the following options to be specified: `fraction`, `n`, `n_positives`.

Note

It is not guaranteed to provide *exactly* the specified fraction/balance/n.

Parameters:

Name	Type	Description	Default
`fraction` Â¶	`float`	Specifies the fraction of instances to be sampled relative to the whole size of the input data.	`None`
`n` Â¶	`int`	Specifies the number of instances to sample.	`None`
`n_positives` Â¶	`int`	Specifies the desired number of instances for the positive class that the sampled data should contain, maintaining the original balance (random sample). If this option is chosen, the parameters `target_field` and `target_value` must also be given.	`None`
`target_field` Â¶	`str`	Name of the target field. This parameter must be specified together with `target_value` if the option `n_positives` was used, otherwise it will be ignored.	`None`
`target_value` Â¶	`str`	Name of class associated with the positive class in `target_field`. It can be a set of classes to represent that all those classes should be kept complete without sampling. This parameter must be specified together with `target_field` if the option `n_positives` was used, otherwise it will be ignored.	`None`
`seed` Â¶	`int`	Random seed for Spark's random number generator.	`None`

See Also

pyspark.sql.functions.rand

TIMESTAMP_MS_FIELD `class-attribute` `instance-attribute` Â¶

```
TIMESTAMP_MS_FIELD = 'fdz_timestamp_ms'
```

seed `property` Â¶

```
seed: int
```

sample Â¶

```
sample(df: _TDataFrame, entity_field: str | None = None, filter_expr: Column | None = None, repartition: bool = False, mark_field: str = MARK_FIELD, optimal_recovery: bool = True, exclude_kickoff_window: bool = True, timestamp_field: str | None = None, recovery_days: int | None = None, entity_fields: list[str] | None = None) -> _TDataFrame
```

Undersample the input data via a history-preserving strategy.

When specifying a filter it may happen that after the filter you get much less data than originally. In this case, your final sample might result in less rows than you expected.

For example, imagine you have 10M rows in your raw data, and you want to work with just 1M. In this case, you would instantiate your sampler with `fraction=0.1`. However, if only 1% of your rows are part of your filter (`filter_expr`) then your sample will only have those rows. In this case the final size will be 1% of the original, instead of 10%.

Parameters:

Name	Type	Description	Default
`df` Â¶	`DataFrame`	Input data.	*required*
`entity_field` Â¶	`str`	Name of the field for which recovery should occur. After random sampling is done, all instances belonging to the sampled entities will be recovered so historical features can be properly computed. Can ony be provided if 'entity_fields' is None and vice-versa.	`None`
`filter_expr` Â¶	`Column`	PySpark expression to filter events from the random sampling election. A typical example is a filter for scorable events.	`None`
`repartition` Â¶	`bool`	Indicate if the output DataFrame is to be repartitioned to match the number of instances per partition of the original dataset. Default is False, meaning default partitioning.	`False`
`mark_field` Â¶	`str`	Name of the field that will be created to preserve the sampling history. Events that were randomly sampled will have a value 'True' for this field. Only applicable if `entity_field` is given. Default is "fdz_sample_ismarked".	`MARK_FIELD`
`optimal_recovery` Â¶	`bool`	Indicates whether optimal recovery should be performed. If set to False, full recovery will be used instead. Only applicable if `entity_field` is given. Default value is 'True'. If 'True', 'timestamp_field' and 'recovery_days' are mandatory.	`True`
	`bool`	Indicates whether first time window should be excluded from marking. Default value is 'True'. If 'True', 'timestamp_field' and 'recovery_days' are mandatory. This is typically useful because profile metrics are not correctly computed in this window.	`True`

Name	Type	Description	Default
`exclude_kickoff_window` Â¶			
`timestamp_field` Â¶	`str | None`	Name of the field that will be used to as timestamp. Required if optimal_recovery is True or exclude_kickoff_window is True.	`None`
`recovery_days` Â¶	`int | None`	Size in days of the window considered. Typically, this value should correspond to the maximum window size of your plan's metrics operator. Applicable if optimal_recovery is True or exclude_kickoff_window is True. In the case that recovery day values are provided through 'entity_fields', this argument is ignored.	`None`
`entity_fields` Â¶	`list`	List of field names for which recovery should occur. Alternatively, can be a list of tuples (field, recovery_days) if different windows are to be applied to each of the fields. After random sampling is done, all instances belonging to the sampled entities will be recovered so historical features can be properly computed. Can ony be provided if 'entity_field' is None and vice versa. In the case that a list of tuples is provided, 'recovery_days' is ignored. If 'exclude_kickoff_window' is 'True' then the corresponding window will be the maximum value of the recovery_days provided in the 'entity_fields' tuples.	`None`

Returns:

Type	Description
`DataFrame`	Sampled data.

Examples:

This examples demonstrates the behaviour of the option exclude_kickoff_window.

```
>>> first_timestamp = df.agg(F.min(timestamp_field)).first()[0]
>>> datetime.fromtimestamp(first_timestamp/1000)
2018-12-20 09:27:53
>>> window_size = 7 #days
>>> df_sampled = sampler.sample(
...     df,
...     mark_field='ismarked',
...     exclude_kickoff_window = True,
...     timestamp_field='timestamp',
...     recovery_days=window_size,
... )
>>> first_marked_timestamp = (
...     df.filter(col(is_marked)).
...     agg(F.min(timestamp_field)).first()[0]
... )
>>> datetime.fromtimestamp(first_marked_timestamp/1000)
2018-12-27 10:15:48
>>> MS_PER_DAY = 24*60*60*1000
>>> first_marked_timestamp >= first_timestamp + max_window*MS_PER_DAY
True
```